

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA1017	Course Title:	Software Engineering
CO Assessed:	CO4	Assessment:	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	Total Marks:	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	Murali Krishnan N	Reg. No.:	192524090
Date:		Duration:	60 minutes

Code of Conduct

I MURALI KRISHNAN N (192524090) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – CI/CD Pipeline Design Exercise

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design a CI/CD (Continuous Integration and Continuous Deployment) pipeline** for the proposed software system to automate the processes of code integration, building, testing, and deployment.

The exercise should include:

1. **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.
2. **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.
3. Identify suitable **tools and technologies** for each stage of the pipeline and justify their selection.
4. Define appropriate **automated testing strategies** to be integrated into the pipeline.
5. Design the **deployment strategy**, including development, testing/staging, and production environments.
6. Incorporate appropriate **security, quality checks, notifications, and rollback mechanisms** in the pipeline.
7. Prepare a **CI/CD pipeline diagram** and explain the workflow from code commit to deployment.

Deliverable: Submit a **CI/CD Pipeline Design document** containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25%)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration(15%)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollback details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation(10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/documentation but lacks clarity	Poorly presented, incomplete, or difficult to understand

Criteria	Mark
Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4
Tools & Technology Selection(20%)	/4
Automated Testing & Quality Integration(15%)	/4
Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4
TOTAL	/25

1. Introduction and Industry Context

Microgrids are increasingly adopted across campuses, industries, and residential communities to strengthen energy resilience and integrate renewable sources such as solar and wind. AI, IoT, smart meters, and cloud computing together enable intelligent energy management, real-time monitoring, and demand-response optimization. Conventional energy management systems lack predictive capability, resulting in inefficient utilization of generated power and higher operational costs.

The AI-Based Smart Microgrid Energy Management and Load Balancing Platform is a cloud-connected software system that monitors electricity generation and consumption, forecasts demand using AI, optimizes battery storage, balances electrical loads automatically, integrates renewable sources, and produces real-time operational dashboards and reports. Because this platform combines an AI/ML forecasting service, IoT data ingestion, a web dashboard, and cloud APIs, it must be released frequently, safely, and with strong quality and security guarantees — making a well-designed CI/CD pipeline essential.

This document analyzes the CI/CD requirements of the platform, designs the complete pipeline workflow from code commit to production deployment, justifies the tools selected for each stage, defines the automated testing and quality strategy, and specifies the deployment, security, and rollback strategy across Development, Staging, and Production environments.

2. Requirement & Pipeline Analysis

Based on the problem statement and expected outcomes (improved efficiency, reliability, and data-driven planning), the following CI/CD-specific requirements were identified for this platform:

ID	CI/CD Requirement	Justification (Why it matters for this platform)
R1	Frequent, reliable releases	Multiple system modules (AI forecasting service, IoT ingestion layer, dashboard, reporting) are updated independently and must be integrated and released without breaking the live microgrid monitoring service.
R2	Automated build & test	Every code commit (backend APIs, AI model code, frontend dashboard) must be automatically compiled/packaged and tested to catch defects before they reach production.

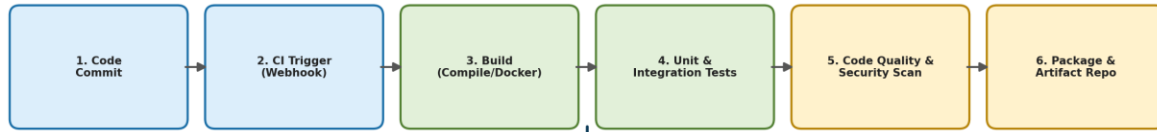
ID	CI/CD Requirement	Justification (Why it matters for this platform)
R3	Data & model quality assurance	AI forecasting accuracy and IoT data-processing logic must be validated automatically (unit + integration tests) before deployment, since incorrect forecasts directly affect battery scheduling and load balancing decisions.
R4	Security & compliance	The platform handles operational energy data and controls physical hardware actions (battery charge/discharge, load shedding), so the pipeline must include dependency, container, and secrets scanning before deployment.
R5	Zero/near-zero downtime deployment	Since the platform provides real-time monitoring and demand-response control, production deployments must avoid service interruption — requiring blue-green or canary deployment support.
R6	Environment parity	Development, Staging, and Production environments must be provisioned consistently (Infrastructure as Code / containers) to avoid "works on my machine" issues.
R7	Traceability & rollback	Every deployment must be traceable to a specific commit/artifact version, with the ability to roll back quickly if a release causes incorrect load-balancing behaviour.
R8	Notifications & observability	The DevOps/engineering team must be notified automatically of build failures, quality-gate failures, and production incidents, with dashboards for pipeline and system health.

3. Pipeline Architecture and Workflow

The pipeline is organized into two phases — Continuous Integration (CI), which validates and packages every code change, and Continuous Deployment (CD), which promotes the resulting artifact through Development, Staging, and Production environments. Figure 1 shows the complete 12-stage workflow.

CI/CD Pipeline — AI Smart Microgrid Platform

Continuous Integration (CI) Phase



Artifact promoted to Dev environment

Continuous Deployment (CD) Phase

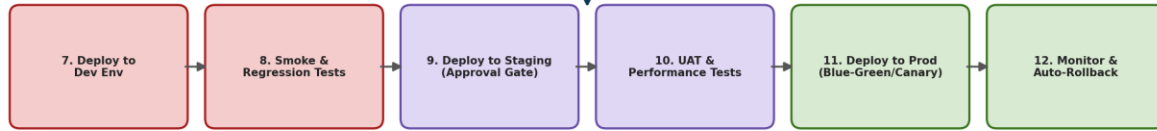


Figure 1: End-to-end CI/CD pipeline workflow for the AI Smart Microgrid Platform

3.1 Stage-by-Stage Description

Stage	What Happens	Purpose
1. Code Commit	Developer pushes code (backend service, AI model, dashboard) to a feature branch.	Triggers the pipeline via a webhook to the CI server.
2. CI Trigger	Source control system detects the push/pull request and notifies the CI server.	Ensures every change is automatically picked up with no manual step.
3. Build	Application is compiled/transpiled; Docker images are built for each microservice (API, AI-model service, dashboard).	Produces a consistent, versioned, deployable unit.
4. Unit & Integration Tests	Automated tests run against business logic, AI forecasting functions, and IoT data-parsing modules; integration tests validate service-to-service communication.	Confirms functional correctness before any further promotion.

Stage	What Happens	Purpose
5. Code Quality & Security Scan	Static code analysis, dependency vulnerability scanning, and container image scanning are executed.	Enforces coding standards and blocks known-vulnerable dependencies from progressing.
6. Package & Artifact Repository	Versioned build artifacts/Docker images are pushed to an artifact/container registry.	Creates a single source of truth for what gets deployed to each environment.
7. Deploy to Dev	The latest artifact is automatically deployed to the Development environment.	Gives developers/testers fast feedback in a live-like environment.
8. Smoke & Regression Tests	Automated smoke tests confirm the Dev deployment is healthy; regression suite re-validates existing features.	Prevents newly introduced defects from silently breaking existing functionality.
9. Deploy to Staging (Approval Gate)	A manual approval step gates promotion; once approved, the build is deployed to Staging.	Adds a human checkpoint before customer/operator-facing validation.
10. UAT & Performance Tests	User acceptance testing and load/performance testing are executed against realistic microgrid data volumes.	Confirms the system meets functional expectations and NFRs (e.g., 5-second dashboard refresh) under load.
11. Deploy to Production	Artifact is released to Production using a blue-green or canary strategy.	Minimizes downtime and limits the blast radius of any undetected defect.
12. Monitor & Auto-Rollback	Post-deployment health checks and monitoring (metrics, logs, alerts) run continuously; automatic rollback triggers on failure thresholds.	Ensures rapid detection and recovery from production incidents.

4. Tools and Technology Selection

The tools below were selected for their maturity, native integration with Git-based workflows, and suitability for a system combining containerized microservices, an AI/ML forecasting component, and IoT data ingestion.

Pipeline Stage/Area	Selected Tool(s)	Justification
Source Control	Git + GitHub	Industry-standard distributed version control with built-in webhook support, pull-request review workflow, and native CI integration.
CI/CD Orchestrator	GitHub Actions / Jenkins	Automates the full pipeline as code (YAML/Jenkinsfile), integrates natively with GitHub, supports parallel jobs, and has a large plugin ecosystem for IoT/cloud integrations.
Build & Containerization	Docker	Packages each microservice (API, AI forecasting engine, dashboard) with its dependencies into a portable, environment-consistent image.
Unit/Integration Testing	PyTest (AI/backend), JUnit (Java services), Jest (dashboard)	Language-appropriate, widely adopted frameworks that integrate cleanly with CI runners and produce machine-readable test reports.
Code Quality Analysis	SonarQube	Provides static analysis, code-smell/duplication detection, and quality-gate enforcement directly inside the pipeline.
Security Scanning	Snyk / OWASP Dependency-Check + Trivy	Detects vulnerable dependencies and scans Docker images for known CVEs before deployment.
Artifact/Container Registry	Docker Hub / Nexus / Amazon ECR	Central, versioned storage for build artifacts and container images that Dev/Staging/Prod all pull from.
Infrastructure as Code	Terraform + Kubernetes (Helm charts)	Ensures Dev, Staging, and Production environments are provisioned identically, reducing configuration drift.
Deployment Strategy Tooling	Kubernetes + Argo Rollouts	Enables blue-green/canary releases with automated health-check-based promotion or rollback.
Monitoring & Logging	Prometheus + Grafana, ELK/EFK Stack	Provides real-time metrics dashboards (matching the platform's own operational-dashboard requirement) and centralized log analysis for fast incident diagnosis.
Notifications	Slack / Microsoft Teams + Email alerts	Immediately informs the DevOps team of build failures, quality-gate failures, or production alerts.

Pipeline Stage/Area	Selected Tool(s)	Justification
Secrets Management	HashiCorp Vault / GitHub Actions Secrets	Keeps API keys, database credentials, and cloud credentials out of source code and pipeline logs.

5. Automated Testing and Quality Integration

Automated testing and quality checks are embedded at multiple points in the pipeline rather than only at the end, so defects, poor code quality, and security vulnerabilities are caught as early as possible ("shift-left" testing).

Testing/Quality Type	When It Runs	What It Validates
Unit Testing	Every commit, in CI build stage	Validates individual functions — e.g., AI forecasting calculations, battery SoC boundary logic, load-balancing rules — in isolation with mocked data.
Integration Testing	After unit tests pass	Confirms IoT data-ingestion service, AI forecasting API, and dashboard backend correctly exchange data end-to-end.
Regression Testing	Before promotion to Staging	Re-runs the full automated suite to ensure new changes have not broken existing monitoring, forecasting, or reporting features.
Static Code Analysis	Every CI build (quality-gate stage)	SonarQube enforces coding standards, flags code smells/duplication, and blocks merges below the quality threshold.
Security/Dependency Scanning	Every CI build (quality-gate stage)	Snyk/Trivy scan dependencies and container images for known vulnerabilities before packaging.
Performance/Load Testing	In Staging, before Production approval	Simulates peak smart-meter node counts and concurrent dashboard users to validate the 5-second refresh NFR under load.
User Acceptance Testing (UAT)	In Staging, with stakeholder sign-off	Validates that dashboards, reports, and demand-response behaviour meet operator expectations before release.

Testing/Quality Type	When It Runs	What It Validates
Smoke Testing	Immediately after each deployment (Dev/Staging/Prod)	Quickly confirms the newly deployed build is up and the critical monitoring/dashboard paths are reachable.

6. Deployment, Security, and Rollback Strategy

The platform is promoted through three progressively stricter environments. Each promotion is gated by automated or manual checks, and every environment can be rolled back independently if a deployed version misbehaves.

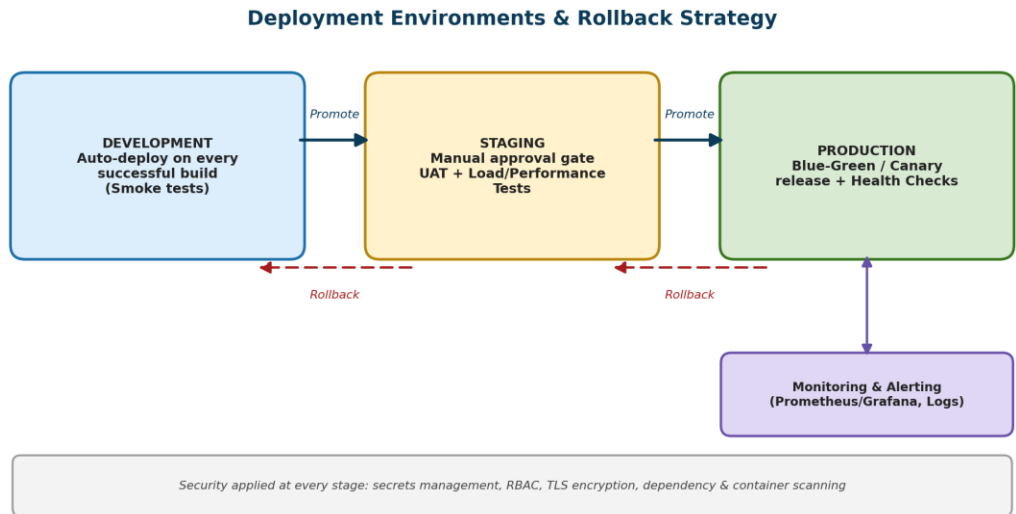


Figure 2: Environment promotion path with rollback and monitoring feedback loop

6.1 Environment Details

Environment	Deployment Approach	Purpose / Safeguards
Development	Automatic deployment on every successful CI build; used by developers/testers for early validation.	Fast feedback loop; no approval gate; smoke tests run automatically after deploy.
Staging	Deployment requires manual approval; mirrors Production configuration (same container images, similar data volumes).	UAT, performance/load testing, and final quality sign-off happen here before Production release.
Production	Released using a blue-green or canary strategy — a new version runs alongside the old version and receives a small percentage of traffic first.	Minimizes downtime and limits impact if a defect slips through; automatic rollback if health checks fail.

6.2 Security Practices

- Secrets management: database credentials, cloud API keys, and IoT gateway tokens are stored in Vault/GitHub Secrets — never committed to source control.
- Role-Based Access Control (RBAC): only authorized roles can approve Staging→Production promotions or access production infrastructure.
- Encrypted transport: all pipeline-to-cloud and service-to-service communication uses TLS.
- Dependency & container scanning: every build is scanned for known CVEs before it can be promoted (see Section 5).
- Immutable, versioned artifacts: the exact image promoted to Staging is the same image promoted to Production, guaranteeing consistency and traceability.
- Audit logging: every deployment and approval action is logged with the responsible user and commit hash for traceability.

6.3 Rollback Strategy

- Every deployment is tagged with its Git commit hash and build number, so any environment can be redeployed to a known-good prior version in minutes.
- Production uses automated health checks (dashboard availability, API response codes, forecasting-service latency); if thresholds are breached within the post-deploy monitoring window, Argo Rollouts automatically shifts traffic back to the previous stable version (auto-rollback).
- A manual 'one-click rollback' is also available to the on-call engineer for incidents monitoring does not automatically catch.
- Notifications (Slack/Teams/email) are sent automatically on any rollback event, build failure, or quality-gate failure so the team can respond immediately.

7. Notifications and Observability

The pipeline integrates notifications and monitoring so the team has continuous visibility from commit to production:

- Build/test/quality-gate failures trigger an immediate Slack/Teams message and email to the responsible developer and DevOps channel.
- Staging and Production deployments post a summary (version, commit hash, approver) to the team channel for traceability.
- Grafana dashboards visualize pipeline metrics (build duration, failure rate, deployment frequency) alongside live application metrics (API latency, forecast-service health, dashboard refresh time).

- Production alerts (health-check failures, error-rate spikes) page the on-call engineer and automatically annotate the incident with the last deployed commit for fast root-cause analysis.

8. Alignment with Expected Project Outcomes

The pipeline design directly supports the expected outcomes defined for the AI-Based Smart Microgrid platform by ensuring every improvement to forecasting, load-balancing, and reporting logic can be delivered quickly, safely, and reliably:

Expected Outcome	How the CI/CD Pipeline Supports It
Improved energy efficiency	Faster, safer releases let AI forecasting and optimization improvements reach production quickly, so efficiency gains are realized sooner rather than sitting in a backlog.
Better load balancing	Comprehensive automated testing (unit, integration, performance) on the load-balancing engine ensures new balancing logic is verified under realistic load before release.
Reduced electricity costs	Blue-green/canary deployment prevents faulty releases from disrupting battery-optimization logic that directly drives cost savings.
Increased renewable energy utilization	Regression testing protects existing renewable-source integration features from being broken by unrelated changes.
Enhanced grid reliability	Automated health checks, monitoring, and auto-rollback minimize the duration and impact of any production issue affecting grid operations.
Sustainable energy management	Consistent, repeatable deployments (Infrastructure as Code) reduce configuration errors that could otherwise cause energy waste.
Data-driven energy planning	CI ensures the AI forecasting/reporting services are always tested and deployed together with the data pipelines they depend on, keeping planning data trustworthy.
Reduced carbon emissions	Reliable, well-tested demand-response and renewable-integration features maximize the effective use of clean energy sources, indirectly supporting emission reduction goals.

9. Risks and Mitigation

The table below identifies key risks specific to automating delivery for an energy-management platform, and how the pipeline design mitigates each one.

Risk	Potential Impact	Mitigation in Pipeline Design
Faulty AI model degrades forecast accuracy in production	Undetected forecasting errors could mislead battery scheduling and load-balancing decisions.	Automated accuracy/regression tests against historical data in CI; staged rollout (canary) limits exposure before full promotion.
Vulnerable dependency or container image deployed	A known CVE in a library or base image could be exploited to disrupt monitoring or control functions.	Mandatory Snyk/Trivy security-scan quality gate blocks promotion until vulnerabilities are resolved.
Downtime during production deployment	Even brief downtime interrupts real-time monitoring and demand-response capability.	Blue-green/canary deployment with automated health checks and instant rollback keeps the previous stable version available.
Configuration drift between environments	Staging validation may not reflect real Production behaviour, causing surprises after release.	Infrastructure as Code (Terraform/Helm) keeps Dev, Staging, and Production consistently provisioned.
Unauthorized or accidental production deployment	A rushed or unreviewed change could be pushed directly into a live energy-control system.	Manual approval gate before Production, RBAC-restricted deployment permissions, and full audit logging.
Delayed detection of a production incident	Slow detection extends the time incorrect load-balancing or forecasting behaviour affects operations.	Continuous monitoring (Prometheus/Grafana) with automated alerting and rollback thresholds.

10. Sample Pipeline Configuration (Illustrative)

A simplified GitHub Actions-style workflow illustrating the stages described above:

```
name: microgrid-platform-ci-cd
on: [push, pull_request]
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build Docker images
        run: docker build -t microgrid/api:$GITHUB_SHA .
      - name: Run unit & integration tests
        run: pytest --junitxml=results.xml
      - name: SonarQube scan
        run: sonar-scanner
      - name: Dependency & image scan
        run: trivy image microgrid/api:$GITHUB_SHA
      - name: Push to registry
        run: docker push microgrid/api:$GITHUB_SHA
  deploy-dev:
    needs: build-and-test
    steps: [ deploy to Dev cluster, run smoke tests ]
  deploy-staging:
    needs: deploy-dev
    environment: staging # requires manual approval
    steps: [ deploy to Staging, run UAT + performance tests ]
  deploy-production:
    needs: deploy-staging
    environment: production
    steps: [ canary deploy, monitor health checks, auto-rollback on failure ]
```

11. Workflow Walkthrough: Commit to Deployment

1) A developer commits a change (e.g., an improvement to the AI demand-forecasting model) and pushes it to a feature branch. 2) The push automatically triggers the CI pipeline, which builds Docker images for the affected services. 3) Unit and integration tests run against the forecasting logic and IoT data handlers; the build fails fast if any test fails. 4) SonarQube and Trivy/Snyk scan the code and container image for quality issues and vulnerabilities; the pipeline halts if the quality gate is not met. 5) On success, the artifact is versioned and pushed to the container registry. 6) The artifact is automatically deployed to Development and verified with smoke tests. 7) After developer/tester confirmation, a release manager approves promotion to Staging, where UAT and performance tests run against realistic data volumes. 8) Once Staging sign-off is complete, the same artifact is released to Production using a canary strategy, gradually shifting traffic while health checks are monitored. 9) If any health check fails, Argo Rollouts automatically rolls back to the last stable version and alerts the on-call engineer; if all checks pass, the canary is promoted to 100% of traffic and the release is complete.

12. Conclusion

The CI/CD pipeline designed above provides the AI-Based Smart Microgrid Energy Management and Load Balancing Platform with automated, secure, and traceable software delivery — from source code commit through build, multi-layered automated testing, code-quality and security scanning, packaging, and progressive deployment across Development, Staging, and Production. By combining shift-left testing, quality gates, blue-green/canary deployment, continuous monitoring, and automated rollback, the pipeline directly supports the platform's own objectives of improved reliability, reduced operational cost, and sustainable, data-driven energy management — while ensuring that new features and AI model improvements can be released frequently without compromising the safety or availability of a system that manages real physical energy infrastructure.